

1. Crew de Agentes

1.1 Justificación

- **Temática:** La aplicación consiste en una productora de contenidos automatizada, específicamente diseñada para investigar y redactar guiones de podcast sobre curiosidades históricas. El tema elegido para la prueba es *"La invención del café y su prohibición en la historia"*.
- **Justificación de Agentes:** Se han utilizado agentes para simular un flujo de trabajo profesional de redacción.
 - Separar la **investigación** (datos objetivos) de la **redacción** (creatividad y estilo) garantiza que el contenido sea veraz pero entretenido.
 - Un solo modelo de lenguaje (LLM) a menudo alucina datos o pierde el tono narrativo si hace todo a la vez. Al tener un agente "Documentalista" y otro "Guionista", cada uno se especializa en su rol, mejorando la calidad final del producto.

1.2 Esquema visual de agentes y flujo de trabajo



1.3 Partes relevantes del código

```
documentalista:
  role: >
    Investigador de Curiosidades Históricas y Datos
  goal: >
    Encontrar anécdotas sorprendentes, poco conocidas y verídicas sobre {tema}.
  backstory: >
    Eres un bibliotecario digital obsesionado con los detalles que nadie más nota.
    Te pasas el día leyendo archivos antiguos y papers científicos.
    Tu misión es alimentar al equipo creativo con "pepitas de oro" de información
    que enganchen a la audiencia desde el primer segundo.

guionista:
  role: >
    Guionista Senior de Audio y Storytelling
  goal: >
    Escribir un guion de podcast inmersivo utilizando los datos del documentalista.
  backstory: >
    Vienes del mundo de la radio y el teatro. Sabes que un buen guion no solo informa,
    sino que entretiene. Tienes una obsesión por el ritmo y la cadencia de las palabras.
    Siempre te aseguras de cumplir con la longitud exacta del texto para no aburrir.
```

- **Explicación:** En la captura se observa la definición de los roles en YAML. Se ha definido un documentalista con un *backstory* enfocado en la precisión histórica y un guionista enfocado en el *storytelling* y el ritmo auditivo.

```
agente_doc = Agent(
    config=agents_config['documentalista'],
    verbose=True,
    allow_delegation=False,
    llm=chatgpt_manager
)

agente_guion = Agent(
    config=agents_config['guionista'],
    verbose=True,
    allow_delegation=False,
    tools=[contador_tool, sentimiento_tool],
    llm=chatgpt_manager
)
```

- **Explicación:** Aquí se instancian los agentes usando la configuración cargada. Es relevante notar que al agente guionista se le asignan las herramientas (`tools=[contador_tool, sentimiento_tool]`) para que pueda auto-evaluar su trabajo.
- **Resultado de la ejecución:** Al ejecutar el sistema, se produce un error de **Rate Limit (429)** por parte de la API de OpenAI. Esto indica que el código intenta conectar correctamente con los servidores de OpenAI para iniciar el proceso jerárquico, pero la cuenta carece de créditos suficientes en este momento.

```
raise self._make_status_error_from_response(err.response) from None
openai.RateLimitError: Error code: 429 - {'error': {'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, read the docs: https://platform.openai.com/docs/guides/error-codes/api-errors.', 'type': 'insufficient_quota', 'param': None, 'code': 'insufficient_quota'}}
```

2. CrewAI Tools

2.1 Descripción de las tools Se han implementado dos herramientas personalizadas:

1. **ContadorPalabrasTool:** Una herramienta que recibe un texto y devuelve el número exacto de palabras.
2. **AnalisisSentimientoTool:** Una herramienta que analiza el texto buscando palabras clave inspiradoras para determinar si el tono es positivo.

2.2 Justificación

- **ContadorPalabrasTool:** En la producción de podcasts, el tiempo es oro. Un guion de 2 minutos debe tener entre 250 y 350 palabras. Los LLMs suelen tener dificultades para contar palabras exactas; esta herramienta programática le da al agente la capacidad de verificar matemáticamente si cumple la longitud requerida.
- **AnalisisSentimientoTool:** Se requiere que el podcast termine con un mensaje inspirador. Esta herramienta permite validar objetivamente si el cierre cumple con los criterios emocionales definidos por la dirección del programa, evitando cierres planos o negativos.

2.3 Partes relevantes del código

```
class ContadorPalabrasTool(BaseTool):
    name: str = "Contador de Palabras"
    description: str = (
        "Útil para contar las palabras de un texto. "
        "Usa esto cuando necesites verificar si un borrador cumple con la longitud requerida."
    )

    def _run(self, texto: str) -> str:
        # Lógica simple de Python
        cantidad = len(texto.split())
        return f"El texto tiene {cantidad} palabras."

class AnalisisSentimientoTool(BaseTool):
    name: str = "Analisis de Sentimiento"
    description: str = (
        "Analiza si un texto tiene un tono inspirador y positivo. "
        "Úsalo para verificar el cierre del guion."
    )

    def _run(self, texto: str) -> str:
        # Buscamos palabras clave que denoten inspiración o positividad
        palabras_positivas = ['futuro', 'esperanza', 'increíble', 'descubrir', 'gracias', 'inspirar', '']
        texto_lower = texto.lower()
        score = sum(1 for p in palabras_positivas if p in texto_lower)

        if score >= 2:
            return "El texto tiene un tono INSPIRADOR y positivo adecuado."
        else:
            return "El texto es demasiado neutro. Intenta añadir palabras más emotivas o inspiradoras."
```

- **Explicación:** Se muestra la creación de clases que heredan de BaseTool. Cada clase define un método `_run` con lógica de Python pura (uso de `len()`.`split()`) para

contar y búsqueda de keywords para el sentimiento), lo que demuestra cómo extender las capacidades del agente más allá de la generación de texto.

- **Resultado de la ejecución:** Debido al error de cuota (429) mencionado anteriormente, los agentes no llegaron a invocar las herramientas en esta ejecución.
- **Resultado esperado:** Si la API tuviera créditos, el agente guionista escribiría un borrador, usaría ContadorPalabrasTool para ver la longitud, y si fuera muy corto, lo extendería. Finalmente, usaría AnalisisSentimientoTool para confirmar que el final es emotivo antes de entregar el resultado final.

3. Proceso de trabajo jerárquico

3.1 Justificación Se ha optado por un proceso jerárquico (Process.hierarchical) porque simula una redacción real con un Editor Jefe.

- En un proceso secuencial, el guionista simplemente tomaría lo que le da el documentalista.
- En el proceso **jerárquico**, el "Manager" (gestor) evalúa la calidad de la investigación del documentalista antes de pasársela al guionista. Si la información es pobre, el Manager podría pedirle que busque de nuevo. Esto asegura una mayor calidad y coherencia en el resultado final.

3.2 Esquema visual



3.3 Partes Relevantes del código

```
# --- CREW ---
def mi_crew():
    crew = Crew(
        agents=[agente_doc, agente_guión],
        tasks=[tarea_buscar, tarea_escribir],
        process=Process.hierarchical,
        manager_llm=chatgpt_manager,
        verbose=True
    )
    return crew
```

- **Explicación:** Se configura `process=Process.hierarchical`. Es fundamental la asignación del `manager_llm=chatgpt_manager`, que utiliza GPT-4o. Este modelo actúa como el "cerebro" que orquesta a los otros agentes, decidiendo qué tarea se ejecuta y validando los resultados intermedios.
- **Resultado de la ejecución:**

```
raise self._make_status_error_from_response(err.response) from None
openai.RateLimitError: Error code: 429 - {'error': {'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, read the docs: https://platform.openai.com/docs/guides/error-codes/api-errors.', 'type': 'insufficient_quota', 'param': None, 'code': 'insufficient_quota'}}
```

Evidencia: El error `openai.RateLimitError` confirma que el sistema intentó contactar con la API para instanciar al Manager. El flujo jerárquico consume más tokens que el secuencial (debido a la gestión del Manager), lo que precipitó el error de cuota. El código es funcional y correcto, cumpliendo con la arquitectura solicitada, a falta únicamente de saldo en la API para completar la generación del texto.